

Module 8 — React: Forms, Props & State

Time: 1 hour · Covers Practical 7 · CO3

Practical 7: Creating a form in React js using components properties (props) & state.

Part A — State: memory that triggers re-render (concept, ~20 min)

Props are data passed *in* from a parent (read-only). **State** is data a component *owns and can change over time*. When state changes, React automatically **re-renders** the component to reflect the new value.

You create state with the `useState` hook:

```
import { useState } from "react";

function Counter() {
  // useState(initialValue) returns [currentValue, setterFunction]
  const [count, setCount] = useState(0);

  return (
    // Clicking calls setCount, which updates state AND re-renders the UI.
    <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>
  );
}
```

Rules:

- Never mutate state directly (`count++` is wrong). Always use the setter (`setCount(...)`).
 - Calling the setter schedules a re-render with the new value.
-

Part B — Controlled form inputs (~20 min)

A **controlled input** is one whose value is driven by React state. The state is the single source of truth; typing updates the state, and the state fills the input.

```
function NameForm() {
  const [name, setName] = useState("");

  return (
    <input
      value={name} // value comes FROM state
      onChange={(e) => setName(e.target.value)} // typing updates state
    />
  );
}
```

For a form with several fields, keep one state object:

```
const [form, setForm] = useState({ name: "", email: "" });
```

```
function handleChange(e) {
  const { name, value } = e.target; // the input's name + what was typed
  // spread the old state, then overwrite the one field that changed
  setForm((prev) => ({ ...prev, [name]: value }));
}
```

Each `<input>` gets a matching `name` attribute so one handler serves all fields.

Part C — Submitting & lifting state up (~15 min)

Handle submit by preventing the browser's default page reload, then doing something with the data:

```
function handleSubmit(e) {
  e.preventDefault(); // stop the full-page reload
  onAddUser(form); // send the data to the parent (a prop callback)
  setForm({ name: "", email: "" }); // reset the form
}
```

Lifting state up: the form lives in a child, but the *list of submitted users* lives in the parent. The parent passes a callback prop (`onAddUser`) down; the child calls it. This is the standard React data flow: **data down (props), events up (callbacks)**.

Read the code: [src/components/UserForm.js](#) (the form + state) → [src/App.js](#) (owns the list, receives new users).

Part D — Validation (~5 min)

Because state holds the values, validation is just checking that state before submitting:

```
if (!form.name.trim()) {
  setError("Name is required");
  return;
}
```

The demo shows an inline error message driven by an `error` state variable.

Part E — Run it

```
cd 08-react-forms
npm install
npm start # opens http://localhost:3000
```

Type a name + email, submit, and watch it appear in the list below — with validation on empty fields.

Summary

- `useState` gives a component memory; changing it re-renders the UI.

- **Controlled inputs** bind `value` to state and update it in `onChange` .
- One state object + one `handleChange` handles a whole form.
- `e.preventDefault()` stops the page reload on submit.
- **Data flows down via props, events flow up via callbacks** (lifting state up).

Now do [practice.md](#) .